*future internet*

MDPI

*Systematic Review*

# Modular Monolith Architecture in Cloud Environments: A Systematic Literature Review

**Lamis F. Al-Qora'n [1,*] and Amro Al-Said Ahmad [2]**

[1] Department of Software Engineering, Philadelphia University, Amman 19392, Jordan
[2] School of Computer Science and Mathematics, Keele University, Keele ST5 5BG, Staffordshire, UK; a.m.al-said.ahmad@keele.ac.uk
* Correspondence: lalqoran@philadelphia.edu.jo

**Abstract**

Modular monolithic architecture (MMA) has recently emerged as a hybrid architecture that is positioned between traditional monoliths and microservices. It combines operational simplicity with modularity and maintainability. Although industry adoption of the architecture is growing, academic research on MMA remains fragmented and lacks systematic synthesis. This paper presents the first systematic literature review (SLR) of MMA in cloud environments. The review follows Kitchenham's guidelines; we searched six major digital libraries for peer-reviewed studies published between 2020 and May 2025. From 369 retrieved records, we included 15 primary studies through a structured review protocol. Our synthesis highlights the problem of inconsistent terminology usage in the literature. It also identifies the architectural scope of MMA, and specifies the adoption drivers such as simplified deployment, maintainability, and reduced orchestration overhead. We also analyse implementation practices—including Domain-Driven Design (DDD), modular boundaries, and containerised deployment—and highlight comparative evidence showing MMA's suitability when microservices introduce excessive complexity or costs. Key research gaps include the absence of consensus on a clear comprehensive definition, limited empirical benchmarking, and insufficient tools support. Thus, this study establishes a conceptual foundation for future research and provides practitioners with structured insights to inform architectural decisions in cloud-native environments.

**Keywords:** modulith; service weaver; domain-driven design (DDD); microservices architecture; software engineering modular monolith; cloud computing



Academic Editor: Paolo Bellavista

Received: 18 September 2025
Revised: 18 October 2025
Accepted: 23 October 2025
Published: 29 October 2025

**Citation:** Al-Qora'n, L.F.; Al-Said Ahmad, A. Modular Monolith Architecture in Cloud Environments: A Systematic Literature Review. *Future Internet* **2025**, *17*, 496. https://doi.org/10.3390/fi17110496



## 1. Introduction

In the last few years, the need to achieve software quality has driven significant innovations in architectural design, specifically in the context of cloud computing [1]. Software architecture plays a foundational role and acts as the basis for shaping key quality attributes such as performance, maintainability, availability, and scalability [2]. Consequently, scholars and professionals have continued to explore architectural patterns and how they can enhance quality in cloud environments.

Among these patterns, microservices architecture has gained popularity in the cloud as it offers scalability, independent development, deployment, and maintenance. Therefore, many companies have started to migrate from monolithic to microservices applications [3]. Because of its tightly coupled components, traditional monolithic applications suffer from the increased effort required to develop and maintain such applications, which makes their

maintenance a hard job [4]. The architectural trade-offs of microservices, including their advantages, have been examined in several empirical studies [5]. However, the architecture suffers from critical disadvantages such as complexity, high operational costs, and the need for sophisticated orchestration, monitoring, and deployment infrastructure [5–7]. Thus, the debate on monolithic and microservices approaches has been continuous in the field of software architecture in both research and practice. Many studies provided valuable insights into the strengths and limitations of microservices; however, they also highlighted the need for exploring alternative architectures that balance modularity with simplicity.

Modular monolithic architecture (MMA) has emerged as a hybrid approach that addresses these limitations. The architecture benefits from the operational simplicity of a monolith while incorporating many other benefits such as internal modularity, maintainability, separation of concerns, improved scalability, and faster deployment cycles, which are always associated with microservices [8]. MMA aims to achieve high cohesion and maintainability without suffering from the distributed system complexities of microservices, and this is achieved by organising a single deployable unit into well-defined, loosely coupled modules with clear boundaries [9]. Despite the recent trend toward a return from microservices to monoliths; the industry has been deeply divided on this issue [10]. A traditional monolithic architecture might include many domains, which could then become modules by applying a Domain-Driven Design (DDD) [11]. Tools such as Google's "Service Weaver" have the potential to facilitate the creation of monolithic applications and deploy them as a collection of microservices [12]. Thus, MMA integrates the advantages of microservices, while simultaneously maintaining the rapid development pace that is typically observed in monolithic applications [13]. Quick development may be achieved by partitioning the system into distinct modules that many development teams can work on individually, independently, and concurrently [14].

The modular monolith concepts share a few similarities with the standard monolith and modularisation techniques, even if they are distinct from both. The combination of microservices and monoliths provides an independent solution to the problems associated with both approaches [9]. Orchestration tools like Kubernetes or Docker are among the many new technologies that have made containerisation feasible. This might facilitate the deployment of modular monoliths in an environment that is cloud-native. When using containers, modules may run independently of one another, which simplifies dependency management and allows scalability while benefiting from having a single codebase.

In industry, there is a growing trend toward re-centralising distributed systems back into modular monoliths, particularly in scenarios where microservice fragmentation has led to degraded performance, increased coordination costs, or over-engineering [9].

The academic research on MMA remains scarce and fragmented even though its adoption is becoming increasingly common in industry. While some case studies and experience reports highlight its advantages, a systematic synthesis of its architectural trade-offs, performance characteristics, migration potential, advantages and disadvantages, impacts on the software industry, and tool support is still insufficient.

This scarcity of previous studies poses challenges for both practitioners seeking to evaluate MMA as an architectural choice and researchers aiming to position them within the broader software architecture body of knowledge. The lack of existing systematic studies means that best practices, trade-offs, and adoption contexts remain poorly understood. To address this gap, we provide a systematic literature review (SLR) on MMA in cloud environments following Kitchenham's methodological guidelines [15], and synthesise existing research to achieve the following:

1. Clarify the definition and scope of MMA in academic contexts.

2. Identify the primary motivations, design principles, and implementation patterns associated with this architecture, as well as the necessary contexts and drivers for adopting or transitioning from/to this architecture.
3. Compare modular monoliths with alternative architectures, particularly monolithic- and microservices-based designs, in terms of empirical evidence of their benefits, challenges, and suitability.
4. Synthesise architectural themes to identify any gaps in current research to suggest areas for further investigations.

Thus, this SLR searches academic databases using a comprehensive and clear search strategy to gain a better understanding of the modular monolith architecture's benefits and drawbacks, as well as the possible configurations for its components. We conducted a systematic review of the literature published from 2020 until the end of May 2025 because of the appearance of the scholarly literature on MMA around this period of time. We screened 306 research papers, from which 15 primary studies were included, one of which was included through the snowballing process. The review is guided by a structured set of research questions, which are introduced in detail in Section 3 (Review Protocol).

We use the term "modular monolithic architecture" to consistently refer to architectural patterns that are also known in the literature and industry as "modular monoliths" or "moduliths". These terms are often used interchangeably; however, we are emphasising the architectural nature and design characteristics of this pattern when we use the term "modular monolithic architecture". This specific clarification is to ensure that clarity is maintained across discussions and comparisons.

The remainder of the paper is structured as follows: Section 2 presents the research background and the literature review. Section 3 presents the research methodology that we utilised in our paper. In Section 4, our results are presented and discussed. Section 5 provides answers to the research questions. Section 6 discusses the results, and finally, Section 7 concludes our research.

## 2. Research Background and Literature Review

This section introduces the literature on MMA and gives some background information on it. To position MMA within the broader landscape of software architecture, this section critically reviews related concepts, definitions, tooling ecosystems, and existing secondary studies. The aim is to establish the industrial relevance of the SLR, and the academic gaps it will address.

### *2.1. Domain-Driven Design (DDD)*

DDD is defined as a software development methodology that addresses complexity in complex businesses and other domains by emphasising the modelling of business requirements and domain concepts [11]. Teams tackle complexity by concentrating on domain knowledge, identifying the most complex and challenging problems with models, and designing the software around those models [11]. To develop many small, business-focused teams and promote agile development, the huge domain model is broken into multiple separate contexts [16]. By implementing these confined contexts as modules, each with separate interfaces, the teams may become even more decoupled from one another and require fewer interactions for operation [17]. This level of modularity might be thought of as a phase of transition between monolithic and microservices architecture [18]. However, assigning the right responsibilities to each module is a challenging endeavour that requires refactoring efforts beginning with the first development phase, which uses a single, poorly understood model [17]. In consideration of this, we can conclude that comprehending the

DDD is necessary when trying to develop modularity in our systems and prepare them for the establishment of a modular monolith or microservices architecture.

### *2.2. The Modular Monolith Architecture (MMA)*

MMA is a hybrid architectural style that integrates the operational simplicity and unified deployment model of traditional monoliths with the internal modularity and separation of concerns that are typically characteristic of microservices [19]. Standard monoliths are characterised by their tightly coupled components. On the other hand, MMA organises code into discrete modules within a single deployable unit, thus enabling modular development without the overhead of distributed systems [20]. This hybrid nature has led to terminological inconsistencies across the literature, with various terms like "modular monolith" and "single-deployment modular systems" used interchangeably.

In MMA, an application is divided into separate, boundary-defined modules or components. Logical boundaries are used to divide the modules into groups with similar functionality. This method greatly enhances the system's cohesiveness. Thus, MMA is the next step in the evolution of monolith architecture since modularity permits a high degree of scalability while avoiding the drawbacks of microservice design [8]. This design has the issue of shared data despite its well-defined module boundaries because the separation only occurs inside the codebase [21]. Figure 1 demonstrates the monolith, modular monolith, and microservices architectures.

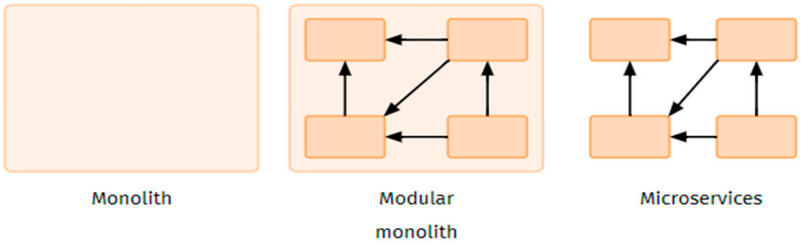


**Figure 1.** Modular monolithic architecture [22].

The authors in [23] declared that the effort should not only focus on the continuing isolation of the monolith, but also the parallel restoration of modularity in the current monolith, where divisions in the modular monolith are always based on the DDD.

In their research, [20] thoroughly reviewed the architecture and emphasised the significance of modular monolith design. Their study highlighted the adoption patterns, motivations for adoption, challenges encountered, and new trends. To find new opportunities, the researchers concentrated on how crucial it is to look at the architecture in more detail, and how emerging trends and technologies like containerisation clouds interact. This emphasises how critical it is to assess recent research on modular monolith and how it interacts with cloud containerisation and other technologies.

An additional study [21] identified MMA as a little-known but possibly useful approach for the adoption of microservices architecture. The researchers emphasised that although practitioners recognise its potential, published studies on its application are scarce. The novel idea of combining the modularity of microservices with the monolithic simplicity of monoliths offers a fascinating direction for further research. It is a great chance for researchers to conduct in-depth investigations that address modulith's theoretical foundation, pragmatic application, advantages, difficulties, and comparative evaluations with alternative tactics for microservices architecture adoption [21]. The researchers also added that extensive record-keeping of practitioners' experiences can provide empirical support for modulith's usefulness, flexibility, and practical applications. Furthermore, the researchers highlighted the importance of investigating its prospective applications and constraints

in various industry areas, which has the potential to offer insightful information on situations in which modulith works well, as well as potential obstacles, and potential remedies for inadequacies.

The authors [24] stated that writing distributed applications entails dividing them up into separately deployable services, and that this architecture has numerous advantages but also many disadvantages. The authors suggested an alternative programming paradigm that avoids these disadvantages. Their approach encourages programmers to create logically split monolithic applications, assign the task of physically distributing and running the modularised monoliths to a runtime, and deploy applications atomically. The authors reported that numerous advantages are opened, and a plethora of innovations are made possible by these three guiding concepts.

Recent work [25] has explored the feasibility of transferring a modular monolith from on-premises to Google Cloud Platform (GCP), with a focus on cost-effectiveness. While their focus lies on migration processes and cost considerations rather than architectural principles, such studies highlight the broader operational contexts in which modular monolithic systems are deployed.

Furthermore, the systematic grey literature review conducted by [9] on the migration process from microservices to MMA highlighted the rising popularity of MMA. The authors reviewed 64 studies and found that the main frameworks used for building modular architecture are Service Waver, Spring Modulith, and Light-hybrid-4j. The study emphasised that in 2023, modular monoliths gained significant focus since there was a substantial increase in the number of studies conducted on the subject. However, the fact that this study—like all grey literature reviews—is dependent on online sources weakens the validity of the findings. Publications from conferences and peer-reviewed publications are more reliable and may increase the reliability of the results. As a result, conclusions that have been made are biased and not supported by actual evidence. To better comprehend the subject and advance the area, a systematic evaluation with an emphasis on academic publications is thus required to enable offering more reliable and thorough information.

Our review of the literature indicates that the advantages and disadvantages of MMA in cloud environments are not receiving much attention. It is thus vital to investigate studies that address these issues and conduct an extensive evaluation of these publications.

The authors in [26] conducted a systematic analysis of factors influencing the adoption of MMA over microservices. However, we found that their review was limited in scope, as it focuses mainly on adoption drivers and relies on a focused search approach (Google Scholar and IEEE, last 18 months) alongside industry case studies and expert opinions. Conversely, our study provides the first comprehensive SLR of MMA in cloud environments; we examine multiple digital libraries over a five-year span, apply a Kitchenham-based protocol, and synthesise definitions, benefits, challenges, comparative analyses, tools, and impacts on quality attributes.

## 3. Methodology

This SLR adopted the three-phase process proposed by [15] for performing systematic reviews in software engineering. The process was designed to ensure transparency, replicability, and comprehensiveness in identifying, selecting, and synthesising relevant literature on MMA. In the planning phase, we found that there is a need for a review because of the scarcity of studies and academic syntheses on MMA in cloud environments. Therefore, we developed a review protocol and defined the research questions, inclusion/exclusion criteria, and the search strategy. Then, we conducted the review, during which we identified 369 studies from electronic databases and 2 studies by using the snowball sampling procedure proposed by [27].

After that, we selected 15 primary studies (one of them came from snowballing) after screening, assessing the quality through inclusion criteria, extracting data using standardised forms, and synthesising the data thematically. Finally, we reported our review in a structured and transparent manner, including PRISMA flow diagrams, to ensure clarity and reproducibility.

### *3.1. Review Protocol*

This section outlines the predefined protocol that we utilised to guide the planning and execution of this systematic review. Our protocol, which follows Kitchenham's guidelines [15], includes clearly formulated research questions, criteria for study inclusion and exclusion, and a detailed search strategy to ensure the identification of relevant and high-quality literature. Developing such a protocol has the potential to ensure consistency, traceability, and transparency throughout the review process.

#### 3.1.1. Research Questions

This systematic literature review attempts to answer the following research questions to help software engineers, architects, and researchers interested in exploring the potential of MMA in cloud environments.

RQ1: How is modular monolithic architecture defined in academic literature?
RQ2: What are the benefits and challenges associated with adopting modular monolithic architecture?
RQ3: What results are available when comparing modular monoliths with other architectural patterns such as traditional monoliths and microservices architectures?
RQ4: What frameworks, practices, and tools are available in the literature and industry to implement MMA?
RQ5: What is the impact of MMA on performance, maintainability, and scalability?

#### 3.1.2. Inclusion Criteria

We specified the inclusion criteria and decided to include studies that satisfy the following conditions:

1. The study explicitly discusses the role of MMA in software engineering. (from an architectural perspective).
2. The study is a peer-reviewed journal article or conference paper.
3. The study is written in English.
4. The study provides empirical data, case studies, or substantial conceptual frameworks relevant to modular monoliths.

#### 3.1.3. Exclusion Criteria

We then defined the exclusion criteria and agreed to exclude studies that satisfy the following conditions:

1. The study focuses exclusively on unrelated architectures (e.g., microservices, event-driven architecture) without direct and focused discussion on modular monolithic architecture.
2. The work is either a non-peer-reviewed study, thesis, tutorial, blog post, or opinion article lacking empirical or theoretical rigour, or else it is a paper not published in a reputable, indexed venue (e.g., not indexed in Scopus or equivalent databases).
3. The study is a duplicate or preliminary version of a later published work.
4. The study has ambiguous findings.
5. The study is a background article.

#### 3.1.4. Search Strategy and Data Sources

We developed a search strategy to search for primary studies and specified the resources to be searched. Scopus, Wiley Online Library, IEEEXplore, ACM Digital Library, ScienceDirect, and Google Scholar were all searched for related articles. Google Scholar was used purposefully as a supplementary source to ensure a comprehensive and trustworthy search for related studies. In other words, while primary databases cover reputable indexed venues, Google Scholar captures emerging works that may not yet appear in the indexed databases. We minimised biases by performing quality screening for all studies that were only sourced from Google Scholar to ensure their eligibility for inclusion.

The period of 2020–May 2025 was selected because it corresponded with a spike in academic research on MMA. Before this time, we conducted a preliminary search for the literature, but we were unable to find any peer-reviewed studies that met our inclusion criteria and specifically addressed MMA, its benefits, drawbacks, or trade-offs.

We formulated the following search string to guide the literature search:

*("Service Weaver" OR "serviceweaver" OR "Spring Modulith" OR "Modulith" OR "modular monolith" OR "modular monolithic") AND ("architecture") AND ("Cloud" OR "Cloud Computing")*

The string was used to search each database, and additional studies were identified by means of snowball sampling, during which we examined references in included papers and forward citations.

#### 3.1.5. Study Selection

The PRISMA flow diagram in Figure 2 demonstrates the study selection process. The following steps were used to identify and select studies:

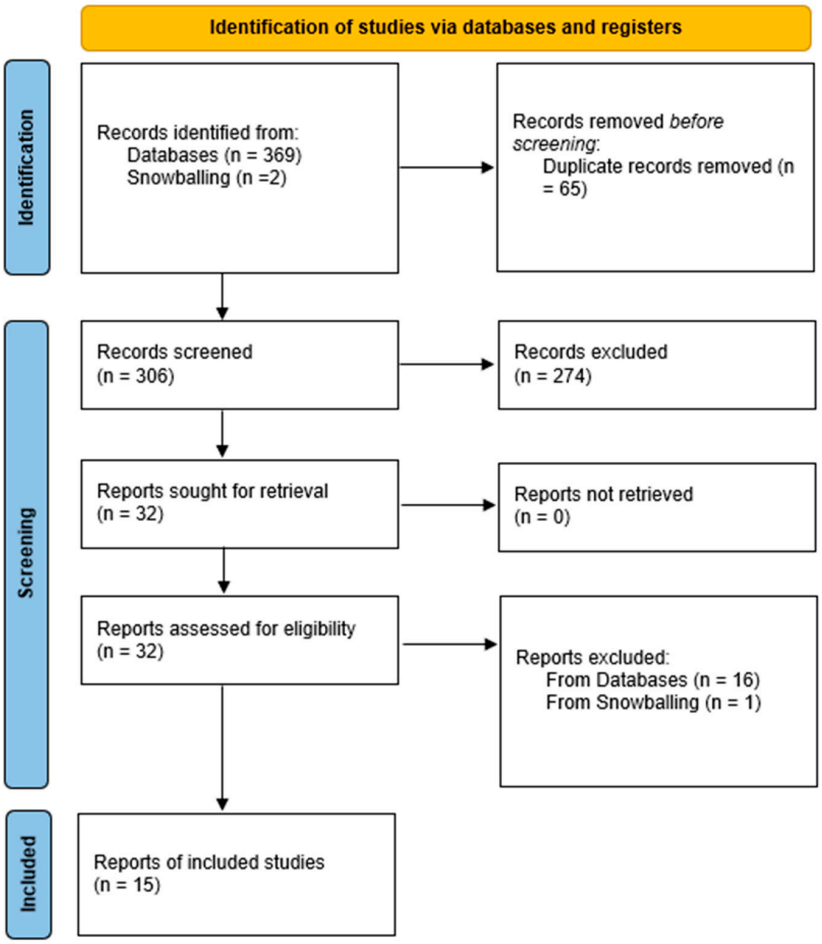


**Figure 2.** PRISMA flow diagram for study selection.

1. Identification: We performed database searches that resulted in the identification of 369 studies through digital libraries and the identification of 2 additional studies through the snowballing procedure. After that, 65 duplicates were removed and 306 studies (304 from digital libraries and 2 from snowball sampling) remained for title and abstract screening.
2. Screening: Titles and abstracts were independently screened against the inclusion criteria by both authors. We excluded 274 studies that did not meet our inclusion criteria. The screening procedure yielded 32 studies (including the 2 obtained from snowballing) to be considered for detailed eligibility assessment.
3. Eligibility: 32 full-text articles were reviewed for relevance through a detailed eligibility assessment. The review process was performed independently by both authors.
4. Inclusion: Final studies meeting all inclusion criteria were included in the synthesis. A total of 15 studies were found, including the 1 study identified through snowballing.

### *3.2. Snowball Sampling Procedure*

Software engineering considers the continual revision of systematic literature reviews to be an essential practice; however, even when the same authors update their reviews, finding new evidence might take a long time [28]. As a result, [28] recommended using search strategies like snowball sampling to assist in the updating of such studies. We used the snowballing procedure suggested by [27] to overcome any limitations with our search string and to ensure that our study is comprehensive.

The procedure of forward snowballing involves authors verifying the papers that cited the papers included in the systematic review [28]. This was carried out to make sure that important studies relevant to our research had not been neglected. Google Scholar was used to find the citations for the papers that were included. Snowballing was performed after the full-text screening, during which we searched for important papers that were highly relevant but missed in the initial search. Since little research is available on modular monolith architecture, we found snowballing to be a beneficial practice. In addition, backward snowballing [27] was conducted to search for any relevant references in the included papers that were not retrieved using our search string.

### *3.3. Data Extraction Protocol*

A standardised data extraction form was developed to ensure consistency. Extracted data included the following:

- Bibliographic details (authors, year, publication venue).
- Study type (empirical, case study, conceptual).
- Architectural characteristics discussed.
- Reported benefits and challenges.
- Tools, frameworks, and technologies mentioned.
- Identified research gaps.

Both authors independently extracted the data and resolved disagreements by discussion.

### *3.4. Data Synthesis Protocol*

A thematic synthesis approach was used to summarise findings for each RQ, with studies grouped by themes (e.g., modularity principles, scalability, maintainability). Quantitative metrics (e.g., publication year trends) were presented where applicable.

## 4. Results

This section presents and discusses our results. The following subsections provide more details:

### *4.1. Identification*

Our search process was initiated by using our search string to search the six identified resources. For accuracy, we followed the inclusion and exclusion criteria specified in the review protocol. We identified 369 articles, of which 65 were removed due to being duplicates. Figure 3 shows the initial search and identification statistics. We conducted the search across the digital libraries demonstrated in Table 1, and the number of initial search results were categorised by database/source. Thus, Scopus, Wiley Online Library, IEEEXplore, ACM Digital Library, ScienceDirect, and Google Scholar were all searched for related articles. We retrieved 369 papers. The Rayyan software, which is designed specifically to help with carrying out systematic literature reviews, is used to upload the retrieved publications. Sixty-five duplicates were removed; these documents could be retrieved from more than one database.

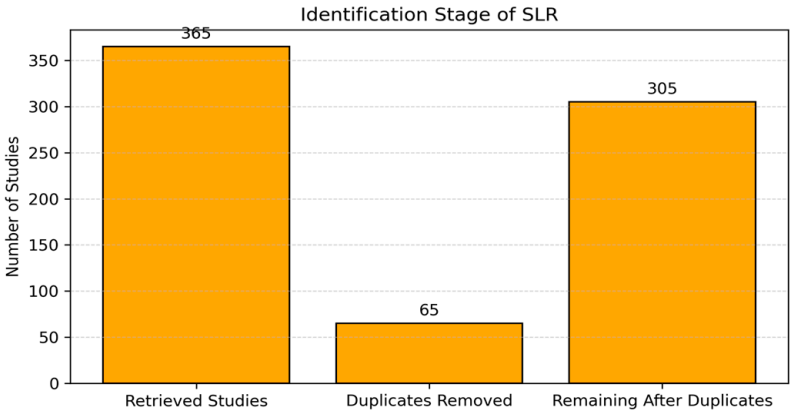


**Figure 3.** Identification stage.

**Table 1.** Number of published papers by database.

| Digital Libraries | Initial Search Results |
|---|---|
| Scopus | 16 |
| Wiley Online Library | 3 |
| IEEEXplore | 30 |
| ACM Digital Library | 27 |
| ScienceDirect | 14 |
| Google Scholar | 279 |
| Other methods: snowballing | 2 |

### *4.2. Screening, Eligibility, and Snowballing Results*

Screening is considered a key task according to Kitchenham's procedure for performing SLR, and it involves an evaluation of the studies that were identified during the initial search. Here, we conducted two stages: title and abstract screening and full-text screening.

After screening (title and abstract screening), we excluded 274 studies that did not meet the inclusion criteria or met the exclusion criteria. This screening process aimed to ensure that only relevant studies were included. Figure 4 shows the screening statistics.

After that, we started the eligibility stage, during which full-text screening was conducted. We excluded 16 studies, as shown in Figure 5. We limited the number of examinable publications to fourteen because these studies explicitly examined the modular monolith architecture in cloud environments in a peer-reviewed publication. This selection process has the potential to ensure the reliability and relevance of the included studies.

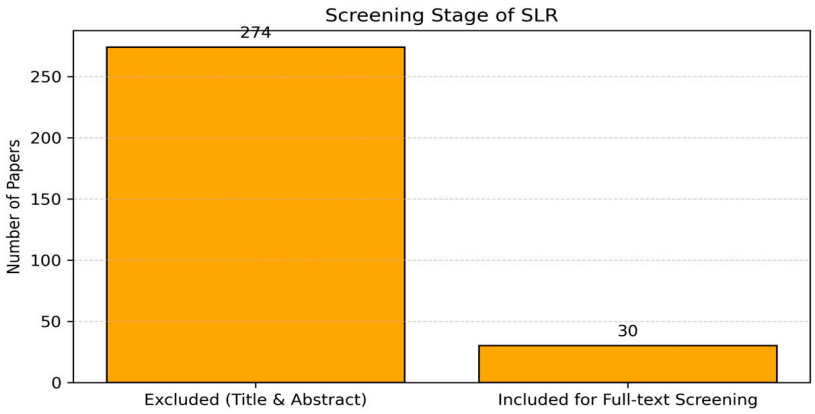


**Figure 4.** Screening.

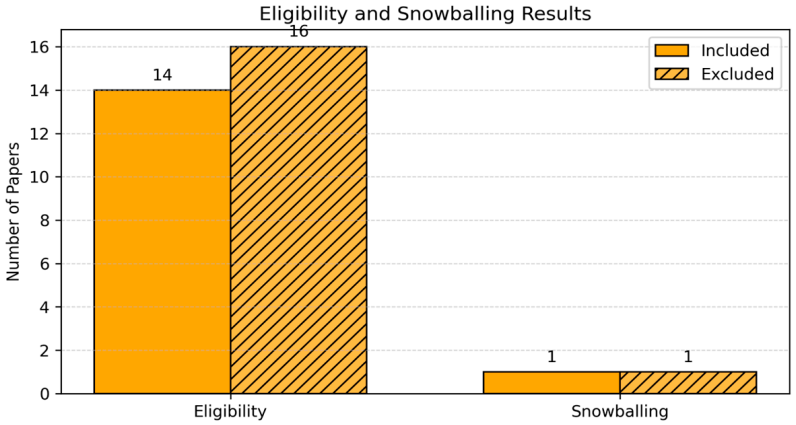


**Figure 5.** Eligibility + snowballing results.

Studies like [25] focused on investigating modular monoliths from cloud migration and platform cost optimisation perspectives and demonstrate that industry interest in MMA intersects with operational deployment decisions.

Papers like [29] do not explicitly address MMA; however, it was included because it provides an empirical analysis of event-driven architectural trade-offs (throughput, latency, and scalability) which offers relevant comparative insights. Such results indirectly inform our discussion of MMA's performance and scalability. This has the potential to ensure that our review captures contextual evidence from relevant architectural paradigms that practitioners might use for comparison when considering MMA adoption.

Snowballing resulted in the retrieval of two papers that were tested for their quality and eligibility for inclusion in the study, and one of them was excluded. We checked all the references for the included papers and the papers that cited them. Figure 5 demonstrates the statistics of the eligible papers included, combined with the snowballing statistics.

### *4.3. Data Extraction*

We followed the previously identified data extraction protocol to ensure consistent and reliable synthesis of the included papers. This protocol captures bibliographic details such as authorship, year of publication, and venue. Then, we extracted data about the study type, in which the studies were classified as empirical, case studies, or conceptual papers. Then, the discussed architectural characteristics of the MMA, the reported benefits and challenges, the frameworks or tooling support, and any stated research gaps were identified. Both authors independently reviewed and extracted data from each study. Inconsistencies in interpretation were discussed and resolved collaboratively, which guarantees reliability in coding and analysis. This structured approach provided the basis for a consistent thematic

synthesis of results across the review's research questions. Furthermore, this structured data extraction served as the foundation for the subsequent synthesis. To derive meaningful insights from the diverse evidence base, we employed a thematic coding process guided by both theoretical frameworks and emergent themes identified during analysis. The following section outlines our hybrid deductive–inductive synthesis strategy in detail.

#### 4.3.1. Synthesis Utilising Thematic Coding Process

As mentioned previously in our review protocol, and according to Kitchenham [15], a key aspect of conducting a systematic literature review is the synthesis process. The study's scope was identified, as previously declared, and all studies that had been published up until May 2025 were taken into consideration. Following that, data was gathered and examined from the included publications. The studies were examined for similarities and differences, and the answers to the study questions were verified by looking through the articles.

A hybrid deductive–inductive approach for thematic analysis was developed. We identified a group of predefined themes informed by the literature in relation to software architecture, modular systems, and cloud computing. Quality attributes from the ISO/IEC 25010 standard [30] were used to help in the analysis. These predefined themes identified:

- Architectural Trade-offs.
- Granularity and Modularity.
- Performance.
- Scalability.
- Cloud Migration and Deployment.
- Tooling and Frameworks.
- Cost and Resource Efficiency.
- Cloud-Native Compatibility.
- Adoption Contexts.

After this deductive framing, we conducted an inductive thematic analysis on the extracted data from each selected paper. Key findings, arguments, or claims were coded under the predefined themes. When new sub-themes or patterns emerged, we incorporated them iteratively into the coding framework.

#### 4.3.2. Coding Procedure and Reliability

Both authors independently read and coded the full texts of all included papers using a shared codebook. To ensure consistency, a pilot round of coding was conducted on five papers. The initial coding results were compared, and any differences in interpretation were discussed. Both authors independently coded all 30 included papers using a shared codebook (snowballing results were assessed separately). To ensure consistency, we performed a pilot round of coding, and the codebook was refined before undergoing full coding. Agreement between the two coders was then assessed using Cohen's Kappa. Out of 30 papers, Author1 marked 14/30 as included and 16 as excluded, while Author2 marked 16/30 as included and 1/30 as excluded. Out of 30 papers, Author1 marked 14/30 as included and 16 as excluded, while Author2 marked 16/30 as included and 1/30 as excluded. Thus, there are only 2 cases of disagreement (author 2 includes, author 1 excludes). This yielded Cohen's Kappa value of 0.87, which indicates almost perfect agreement according to [31] as it exceeded the 0.75 thresh-old, which indicates the degree of resemblance of the subjects. Any minor disagreements were resolved through discussion until consensus was reached, thereby ensuring systematic and reliable coding.

This process ensured that the coding was systematic, replicable, and minimised potential researcher bias, thereby strengthening the validity of our findings.

#### 4.3.3. Thematic Saturation

We decided that we had achieved thematic saturation [32] when we could not identify any new codes or sub-themes after further analysis of three papers in a row. Thus, all major conceptual categories relevant to the research questions had been well represented, and the themes consistently appeared across studies of different contexts, time periods, and publication types.

### *4.4. Data Synthesis*

Table 2 summarises the conclusions of the synthesis process. Moreover, the studies that resulted from the snowballing process were also synthesised and analysed, and the analysis results were added to Table 2.

**Table 2.** The conclusions of the synthesis process.

| Theme | Benefits | Challenges | Representative Studies | Type of Evidence |
| --- | --- | --- | --- | --- |
| Architectural Trade-offs | - Encapsulation and cohesion improve maintainability [32–34], understanding codebase is easier and reduces coordination overhead [33].<br>- Reduced complexity, and simplified deployment processes [34].<br>- Some studies provided conceptual and empirical evidence of monolith vs. microservices and highlighted the importance of adopting modular monolithic designs [35].<br>- For teams with little experience in microservices, adopting an MMA can help build modular thinking without requiring steep learning efforts [33].<br>- Supports DDD-based design, enhances maintainability [22,33,35,40].<br>- Clear module boundaries enable separation of concerns [24,34,38,40].<br>- Achieving balance between the modularity of microservices. and the simplicity of monolithic deployment [41].<br>- Enabling modularity and supporting reuse [26] and easier maintenance if well-defined boundaries are identified [40].<br>- Supports future migration to microservices [41]. | - Risk of hidden coupling.<br>- Larger-scale projects, as the monolithic architecture may result in a larger codebase when compared to the microservice [22].<br>- Harder to implement in big systems [22]. | [17,22,24–26,33–39] | Empirical, case studies, conceptual, prototype evaluation [24]. |
| Granularity and Modularity | | - Refactoring effort required [35] | [22,33,34,38,40,41] | Empirical + conceptual case studies. |
| Performance | - In-process calls reduce latency [35].<br>- Less overhead than network-based microservice communication.<br>- Improved performance [24,29,35]. | - Bottlenecks under heavy load.<br>- Lacks built-in resilience mechanisms.<br>- Performance challenges due to the service communication between different modules within the modular monolith [40]. | [17,29,34,35,38,40,41] | Benchmarks + empirical case studies. |

**Table 2.** *Cont.*

| Theme | Benefits | Challenges | Representative Studies | Type of Evidence |
| --- | --- | --- | --- | --- |
| Scalability | - Suitable for low-to-medium scale deployments.<br>- Consistent performance in bounded contexts. Good vertical scaling [41].<br>- Flexibility, faster development, and scaling [33,34,41]. | - Cannot scale modules independently.<br>- Elasticity requires re-architecture.<br>- Scalability challenges in larger projects and under heavy loads.<br>- Scalability limitations of MMA when compared to microservices architecture [17]. | [17,22,24,29,39–41] | Conceptual + benchmarks. |
| Cloud Migration and Deployment | - Supports phased decomposition through architectural patterns such as the Strangler Fig, allowing for gradual and controlled refactoring of legacy systems.<br>- Enables a gradual, low-risk transition from monoliths to microservices by supporting incremental refactoring [40] migration cost [39].<br>- MMA provides an intermediate phase toward a full microservices architecture [17,35,38–40]. | - Coupled legacy systems make modularisation complex.<br>- High refactoring effort [35].<br>- Concerns related to challenges when migrating to microservices [35]. | [17,25,26,33,35,38–40,42] | Case studies + migration documentation. |
| Tooling and Frameworks | - Frameworks (e.g., The Spring Modulith (Spring, 2024) framework) [34] introduce an experimental approach for developing modular monolith applications within the Spring framework.<br>- The Service Weaver [24,33] simplifies modular design. The Springboot framework was utilised in [32,34].<br>- IDE support enhances testing and maintainability. | - Immature ecosystem.<br>- Lacks universal tooling standards. | [22,26,33,34,39,40] | Conceptual + prototype implementations. |
| Cost and Resource Efficiency | - Reduced infrastructure and ops costs [24].<br>- Avoids network overhead and complex DevOps.<br>- Easily containerized for CI/CD. | - Initial modularization effort.<br>- May lead to inefficiencies at scale. | [17,25,39,40,42,43] | Empirical migration + cost analysis. |
| Cloud-Native Compatibility | - Supports single-deployment unit with modularity, emphasises container orchestration, deployment abstraction, and boundary isolation. | - Limited orchestration control.<br>- No per-module scaling or resilience. | [24,25,33,34,36,38–40,43] | Empirical + conceptual. |
| Adoption Contexts | - Ideal for DDD-mature teams.<br>- Best for systems with existing monoliths and moderate scaling needs. | - Poor fit for globally distributed teams or greenfield systems requiring autonomy. | [22,25,26,35,39–41] | Case studies, architectural reflections. |

Figure 6 displays the analysis of the 15 included studies. When these papers were analysed by year of publication, it became clear that a new trend started to take place when 2023 witnessed an enormous spike in publications. As shown in Figure 6, the first scholarly article on MMA was published in 2021. We also spotted seven articles in 2023, another six articles in 2024, and one article up until May 2025, which illustrates the significance of the subject and demonstrates the extent to which it is growing.

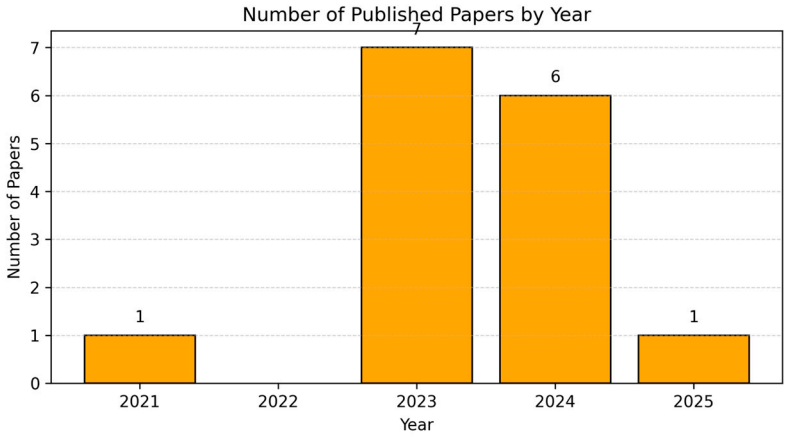


**Figure 6.** Number of published papers by year.

As we can see from Figure 7, 14 papers applied empirical research, while 1 article did not involve empirical methods. Thus, as mentioned previously, we included sixteen papers during our review of the literature. We found two papers during the backward snowballing process [24,44]. The performance of the monolith and intermediate phases was compared to that of microservice architecture by [44], who developed a multi-stage architectural migration into microservices architecture. The authors found that the latency of the microservice design is higher than that of both the monolith and the modular monolith design. This highlights a key advantage of modular monolithic design, which is its ability to surpass microservices architecture in terms of performance, as discovered in our comprehensive review. Although this paper contains significant information, we decided not to include it since it was written in a language other than English and because it does not discuss MMA directly, and we decided to include paper [24].

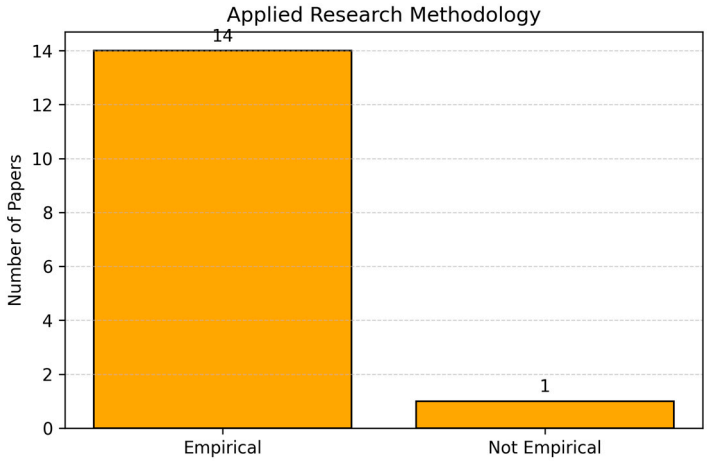


**Figure 7.** Applied research methodologies.

## 5. Answering the Research Questions

### *5.1. RQ1: How Is MMA Defined in Academic Literature?*

MMA is defined across some of the included studies as a single deployable unit that is internally organised into distinct, loosely coupled, and cohesive modules with clear boundaries [22,43]. Unlike traditional monoliths, which are often tightly coupled and difficult to maintain, MMA emphasises separation of concerns and domain-driven modularisation within a unified codebase as discussed in [35]. This modularisation is achieved by applying concepts from DDD, where domains are partitioned into bounded contexts that can evolve independently while still belonging to the same deployable artefact.

Most studies, such as [34], compared MMA with microservices architecture. MMA utilises internal modular boundaries while maintaining the operational simplicity of a monolith [34,35,43]; in contrast, microservices express modularity into independently deployed services. This helps in reducing distributed system overheads such as communication latency and orchestration complexity, while still supporting maintainability, scalability, and faster development cycles [35].

Some studies such as [17,34,35] see the modular monolith as an intermediate artefact and a middle ground approach [36]. This is because it has the potential to facilitate the migration process of a monolith to microservices architecture by separating concerns.

The paper [29] focused on event-driven architecture and did not provide a definition of MMA. Its focus was on evaluating performance characteristics of event-driven systems rather than MMA-specific design principles.

However, there is inconsistency in using the terminology in academic and industry sources. Studies interchangeably use the terms "modular monolith" [22,34,35,43], "modular monolithic architecture" [22,34], and "modular monolith architecture" [34]. Some papers also emphasise MMA's hybrid nature, presenting it as a "middle ground" between the rigidity of monoliths and the complexity of microservices [35]. Despite this variation, the core consensus is that MMA represents a design and deployment style where modularity is enforced at the architectural level within a monolithic boundary.

After exploring all these perspectives, we provide a comprehensive definition for MMA:

MMA is a single deployable software system that enforces modularity at the architectural level by organising the codebase into cohesive, loosely coupled, and domain-driven modules with explicit logical boundaries. This is different from traditional monoliths, as MMA emphasises separation of concerns to enhance maintainability. This is also beneficial as it helps to avoid the orchestration and network overhead of microservices. Unlike microservices, MMA provides logical modular boundaries rather than physical boundaries. It also exists within one deployable unit rather than across distributed services. This hybrid nature helps in proving that MMA is a middle-ground between monoliths and microservices. It offers operational simplicity alongside structured modularity. This helps the architecture in serving either as a stable long-term solution or as a transitional architecture within cloud-native environments.

### *5.2. RQ2: What Are the Benefits, and Challenges Associated with Adopting MMA?*

The 15 primary studies were analysed, and the results are shown in Figure 8. Many themes emerged regarding the benefits and challenges of adopting MMA.

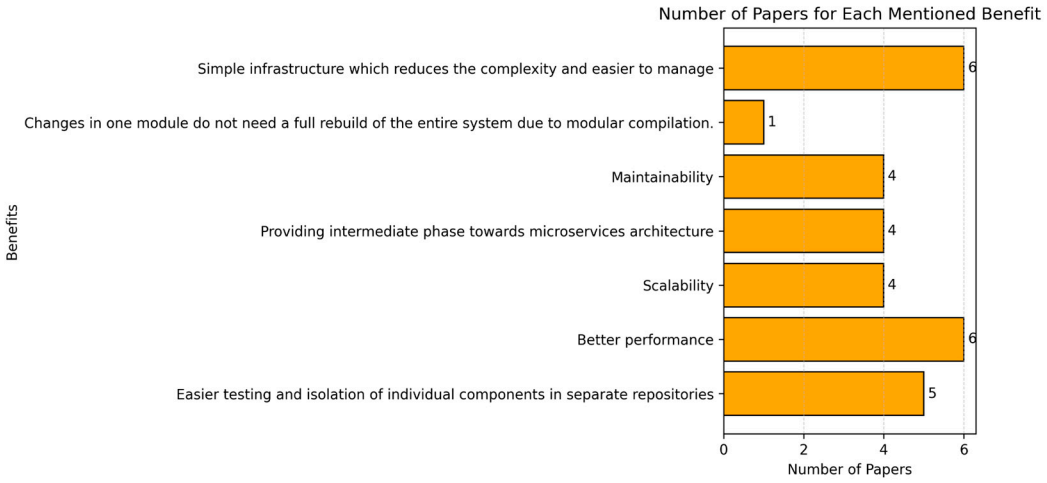


**Figure 8.** The number of papers that mention each benefit.

5.2.1. Benefits of Adopting MMA

1. Simplicity and Reduced Architectural Overhead

MMA simplifies system design and deployment due to its single runtime environment and unified build process, where the infrastructure is simple and has no complexities [17,22,33,34,38,41]. Having a single runtime environment and modular compilation implies that changes in one module do not require a full system rebuild [22]. Furthermore, managing business logic is easier as the modular monolith facilitates business logic, enables close interconnectedness of entities, and accordingly allows for a comprehensive domain model [40]. The authors in [33] also conducted an empirical study in which they discussed the design, development, and testing of cloud-based ERP systems. Although the authors do not explicitly mention the benefits of the modular monolith in the cloud, some benefits can be concluded, such as the ERP system being divided into clear modules, which makes it easier to manage the core business processes in a modular, organised manner.

Keeping a single deployable unit reduces DevOps overhead, which speeds up and simplifies delivery. This simplicity reflects the cost-complexity trade-off. Furthermore, it is evident that the debugging process is considerably simpler when compared to microservices.

2. Maintainability and Modularity

Since all domain model components remain stored in the same database, MMA allows for transactional execution of the monolith functionalities while facilitating small teams, with one for each part of the domain model. This leads to improved structure and improved code maintainability and readability [24,35], and these factors were also mentioned by [41] as MMA simplifies development and maintenance and reduces orchestration between services. Thus, the MMA enables modularity and reusability and easier maintenance if well-defined boundaries are identified [17,33–35,40]. They also added that using the modular approach (REST API testing of CRUD operations) enables easier testing and isolation of individual components, which promotes maintainability [33]. Finally, in [41], the authors discussed the benefits of using modular monolith with the Service Weaver framework, stating that modular monolith simplified development and maintenance by focusing on business rules within a single and unified application, leading to reduced orchestration between independent services and making the application easier to maintain.

MMA enhances code quality only when limited conditions are clearly defined and architectural discipline is maintained, which makes maintainability benefits conditional. Without controlled design, hidden coupling may reduce these benefits.

3. Gradual Migration and Reusability

MMA supports the gradual migration to microservices, which is enabled by clearly defining modular boundaries and separating concerns, thereby decreasing dependencies between various components. This helps in the extraction of individual microservices and provides a groundwork and a transitional stage before a complete microservices architecture is implemented [17,35,38–40]. A well-structured modular monolith has clear domain boundaries that facilitate future migration, leading to faster delivery of functionality [38]. The authors of [38] provided a conceptual discussion on transitioning from monoliths to microservices, with a particular focus on modular monoliths as an intermediary architecture. They stated that having a single deployable artefact minimises the infrastructure complexity when compared to microservices. Therefore, MMA allows teams to migrate gradually to microservices while preserving stability.

4. Performance and Efficiency

Performance optimisation was one of the benefit themes we concluded, as optimised access to domain entities, which is enabled by the modular monolith, has the potential to

improve performance for some functionalities [35,41]. Also, performance improvements in some cases are noted when compared to traditional monolithic architecture (for example, when unique identifiers are introduced to speed database access) [17]. The authors in [34] conducted an empirical study and focused on the practical aspects of implementing and evaluating Service Weaver (a framework for building cloud-native applications using modular monolith architecture). They found that MMA achieves faster communication via method calls without the need for service discovery, which reduces the complexity of communication and accordingly increases performance compared to microservices architecture [34]. Applications can benefit from faster method calls that are less prone to failures [34]. Also, having direct method calls without interservice communication overhead improves performance [17,24,38]. The authors in [24] provided considerable evidence that MMA can deliver superior performance and efficiency compared to microservices. Finally, as stated by [17], reducing overhead communication was one of the core advantages of eliminating the overhead associated with inter-process communication between the services. Thus, as components are co-located in the same process, communication happens within the same memory space, which means that reducing the latency of services leads to improved performance, especially under high loads.

Although MMA was not explicitly addressed in the paper [29], it did highlight the performance-related benefits and challenges of event-driven systems, providing helpful insights into the trade-offs that MMA attempts to address, particularly regarding communication overhead and system responsiveness.

To conclude, MMA facilitates faster communication in exchange for reduced elasticity. However, performance benefits decrease at higher levels, where single-process limitations emerge.

## 5. Flexibility, Deployment, and Cost Optimisation

MMA could potentially be able to control the level of service granularity [17,22,33,35,38]. From a technical point of view, the abstraction allows developers to work on individual modules, each of which can be developed in a separate repository branch using a set of GitHub actions that generate the module artefacts and the application's final artefact/image automatically [22].

More flexibility [17], faster development, and scaling are crucial for adapting to business needs in the cloud environment, and this is enabled by MMA [40]. Moreover, Ref. [41] ensured the flexibility as a core benefit, which enables the deployment team to adjust the granularity of services dynamically without the need for code change, which has the potential to lead to scaling the system more easily based on demand and support resource optimisation and cost savings. Thus, their paper concluded that modular monolith offers a balance between the simplicity of traditional monolith and the scalability of microservices [34].

Simple deployment process because all of the components are packaged together, and deploying to an environment like Google Kubernetes Engine (GKE) is possible with a single command [34]. Moreover, general cloud migration leads to cost optimisation, as using MMA can result in significant savings compared to on-premises infrastructure [39,42]; specifically, MMA lowers infrastructure management costs. This was proved by [39] using experimental data. In terms of scalability and flexibility benefits, these were discussed by [39,40,42]. MMA enables better scalability when compared to traditional monolithic architecture. The benefits of reusing existing licences were also mentioned in [42]. Finally, modular monolith architecture creates a refactoring-friendly environment, allowing developers to make changes and optimise code [17].

In conclusion, MMA is financially and operationally efficient for small-to-medium enterprises or for applications in which elasticity is not a core requirement.

#### 5.2.2. Challenges of Adopting MMA

1. Complexity and Boundary Definition

MMA has many concerns in larger-scale projects because it may result in a larger codebase than the microservices (challenges with feasibility in bigger projects), which have lots of branches that are hard to keep up with [22]. Also, version control systems struggle with managing modular monoliths in large-scale systems [22]. Moreover, in large-scale applications, it is not easy to identify clear component boundaries in modular monoliths [40].

Furthermore, the architecture suffers from complexity problems related to redesigning functionalities to support coarse-grained interactions and to manage dependencies between modules [35]. Moreover, the number of relationships among the new modules' domain entities determines the migration operation costs [35]. The authors in [40] talked about the complexity of maintaining a monolithic application. While the modular design might eliminate such issues, it is challenging to build well-modular applications and avoid dependencies [24]. Thus, in large-scale applications, identifying clear component boundaries in modular monoliths is not easy. The structural complexity of building a well-structured modular monolith was highlighted by [38], who stated that it is difficult to build clear boundaries because it is not easy to separate domains and subdomains. Moreover, there is complexity in defining clear APIs that facilitate future microservice extraction through modularisation [38]. Thus, modularity does not automatically translate into practical modularity.

2. Performance Bottlenecks in Complex Systems

Performance was also another challenging issue. The authors in [17] stated that moving large amounts of information between modules via Data Transfer Objects (DTOs) leads to performance overhead. Also, the authors in [17] added that interconnected modules with synchronous dependencies between modules may lead to tightly coupled modules, which can affect performance when migrating to microservices. Moreover, the authors in [35] think that MMA has a clear impact on performance where performance issues can be seen when the volume of data increases. Furthermore, the authors in [40] mentioned performance challenges because of the service communication between different modules within the modular monolith in comparison to traditional monolithic architecture. However, as mentioned in the benefits theme, the MMA provides better performance when compared to the microservices architecture. Performance issues in cloud environments are usually caused by poor design [38]. Finally, papers [17,24] agreed that interconnected modules with synchronous dependencies between modules produce tightly coupled modules, which can cause performance problems when migrating to microservices. Furthermore, paper [24] highlighted important limitations regarding infrastructure dependency.

To conclude, the performance of the MMA depends heavily on the local deployment. When it comes to distributing modules, the architecture's main goal is weakened.

3. Refactoring and Migration Efforts

A significant refactoring effort is involved in converting the monolith to a modular monolith [35]. The authors in [17] conducted an empirical study, providing details on a case study that involves the migration of a large object-oriented monolith (LdoD Archive) to a modular monolith and then to a microservices architecture. An initial refactoring effort is required to achieve the desired modularity for later migration, particularly in complex systems [17]. Improper modularisation has the potential to complicate future transitions [38].

The high costs required to adopt MMA may prevent its adoption. In other words, teams and organisations with less refactoring experience or projects with short time frames may find the initial technical costs required to adopt MMA unreasonable.

4. Scalability and Elasticity Limitations

Scalability is considered a main benefit of MMA in comparison with traditional monolithic architecture; however, there are many scalability challenges for MMA for larger projects [22]. Thus, MMA may still have scalability limitations when compared to the microservices architecture [17,24].

In [34], the authors reported that resiliency patterns are not built-in where developers are responsible for handling failures, timeouts, and other fault-tolerance mechanisms when using Service Weaver. They added that no specific security solutions are provided. Other challenges that the authors mentioned were specific to Service Weaver and not to the MMA itself.

MMA follows a constrained scalability model, where the scalability is constrained by its single-process deployment, making it effective only for moderate workloads.

The authors in [33] do not explicitly discuss the challenges of the modular monolith in the cloud; however, some challenges can be concluded from the described process. The ERP system's development time can be challenging because of the complexity of modularising intricate business logic. The authors highlighted the importance of extensive unit testing and API testing to ensure the integrity of each module. This can be challenging in a cloud-based environment [33].

Finally, as stated by the authors in [41], while modular monolith architecture offers a good balance between scalability and simplicity, it comes with critical challenges. Communication overhead becomes significant when modules are distributed across different processes or virtual machines, leading to performance degradation. Moreover, the authors confirmed that scaling under heavy loads can be difficult due to the monolith's centralised nature, and managing the balance between loosely coupled modules also introduces additional challenges.

In general, MMA's benefits and challenges demonstrate a rebalancing of architectural trade-offs. When compared to distributed autonomy and scalability, MMA places more emphasis on local modularity and maintainability. It demonstrates a concept of constrained modularity, in which modular cohesion and structure can be achieved without distribution. In summary, its advantages are its ease of use, performance effectiveness, maintainability, and gradual migration. On the other hand, limited resilience, refactoring effort, modularisation complexity, and scalability are among its challenges.

This synthesis highlights the unique nature of a MMA architectural approach, which is designed for cost-effectiveness, maintainability, and simplicity under constrained operational contexts rather than just being a temporary solution towards microservices.

### *5.3. RQ3: What Are the Available Results When Comparing Modular Monolith with Traditional Monolithic and Microservices Architectures?*

Table 3 compares monolithic, microservices, and modular monolithic architectures, providing a comparative overview that highlights how the modular monolithic architecture balances the simplicity of monoliths with the benefits of microservices.

Tsechelidis et al. [22] provided a comparison with microservices showing that microservices architecture is preferred over MMA specifically to avoid the complexity of large-scale projects. No comparison was provided in [39], in which the main focus is on the cost and feasibility of migrating a modular monolith to various cloud models. Also, papers [34,42] did not provide any comparison.

**Table 3.** Comparison of monolithic, microservices, and modular monolithic architectures.

| Aspect | Monolithic | Microservices | Modular Monolithic (MMA) |
| --- | --- | --- | --- |
| Performance | Fast local execution but limited under heavy load. | Network overhead from remote calls. | Local in-process calls reduce latency; efficient for moderate workloads. |
| Scalability | Vertical scaling only. | Independent horizontal scaling. | Bounded scalability: effective for moderate workloads but limited by single-process deployment. |
| Complexity | Simple deployment; fixed structure. | High operational and orchestration complexity. | Lower complexity than microservices; modular discipline required. |
| Maintainability | Hard to refactor or test. | Independent service evolution. | Easier maintenance via DDD-based modularity and clear boundaries. |
| Cost and Resources | Low infrastructure cost. | High infrastructure and DevOps overhead. | Balanced cost profile; suitable for small or medium-scale systems. |
| Evolutionary Role | Legacy baseline. | Mature distributed paradigm. | Transitional or hybrid architecture bridging the two ends. |

In his paper, Barde [40] utilised secondary data from companies like Shopify, Root, and Google and analysed this data to provide insights. This descriptive and analytical study provided a good comparison; in it, he stated that applications with moderate traffic can benefit from monolithic architecture's advantages in performance and reduced latency, which can be attributed to its single, self-contained codebase. However, it is not recommended for systems that undergo frequent changes because of its restricted scalability and update flexibility. He added that individual services benefit from the flexibility, scalability, and independence offered by a microservices architecture. Also, it is resilient and does exceptionally well in supporting increased loads. However, it complicates network communication and security, which makes it difficult for specific applications. The authors stated that MMA reduces the complexity related to microservices while maintaining the advantages of a thorough domain model and modularisation. This has been successfully implemented by companies such as Shopify, Google, Root, and Euro commercial Properties (Estatio).

The authors in [34] compared the modular monolith (Service Weaver) with microservices. It states that Service Weaver enables simplified deployment, communication, and observability but lacks some of the resilience and polyglot support that microservices architectures offer. It also contrasts Service Weaver's simplicity and ease of use with the complexity and flexibility of microservices, particularly in distributed systems; however, the paper does not provide any direct comparison with a traditional monolith.

Shablii and Sergiy [38] highlighted the benefits of a modular monolith over a traditional monolith, emphasising its clear domain boundaries and the potential for easier migration to microservices. Their paper contrasts microservices with modular monoliths. It mentions that while microservices offer better scalability, resilience, and technology diversity, they also come with significant complexity in terms of infrastructure, data management, and deployment orchestration. Modular monoliths are presented as an intermediary option that retains the simplicity of monoliths while preparing the system for future scalability through microservices.

Faustino et al. [17] make a comparison between a typical monolith, a microservice architecture, and a modular monolith. They emphasise the advantages and disadvantages of utilising a modular monolith as a transition step between the two. Although it is associated with its own set of performance problems, the modular monolith is considered an intermediate structure that can assist in balancing the challenges of moving straight from a monolithic to a microservice architecture. The performance degradation caused by

microservices due to network overhead and distant interaction is discussed in the paper. This problem is less common in modular monoliths, but can still cause problems depending on how much data is transmitted between the modules.

The study [29] indirectly answers this question by comparing monolithic and event-driven architectures, which provides comparative insights into architectural trade-offs. These findings establish a benchmark for evaluating MMA's performance and scalability in comparison to other patterns.

The authors in paper [24], which resulted from the backwards snowballing, discussed splitting any application into discrete services for separate deployment. However, they criticised this strategy, stressing that a microservices-based design frequently causes harm and creates difficulties that outweigh the advantages the architecture seeks to achieve. This is essential because microservices confuse physical (how code is delivered) and logical (how code is written) boundaries. To address these issues, they offered up an alternative programming style that decouples the two. Using this approach, developers create their applications as logical monoliths, assign the responsibility of distributing and executing programmes to an automated runtime, and deploy programmes atomically. Compared to the current state, their prototype approach reduces costs by up to 9 times and lowers application latency by up to 15 times. Thus, their paper compared monoliths, microservices, and modular monoliths. They concluded that modular monoliths combine the simplicity of monoliths with some of the scalability benefits of microservices. Decoupling logical and physical boundaries eliminates much of the overhead caused by interservice communication in microservices. However, the paper retains some of the deployment challenges of monoliths, such as needing to redeploy the whole system for specific changes.

The comparative evidence from the included studies indicates the following:

1. MMA reduces communication and operational overhead of microservices. On the other hand, traditional monoliths do not show any modular structure and maintainability.
2. MMA works better in moderate workloads. Single-process deployment limits horizontal scaling while ensuring low latency and simpler management.
3. Most studies viewed MMA as a transitional stage, an intermediate artefact, and an emerging middle-ground before transitioning to microservices [35,40].

In other words, MMA represents a modular architecture that provides balanced simplicity, lower communication, maintainability, and cost efficiency while avoiding the complications of fully distributed systems. The architecture suffers from scalability issues.

Thus, monoliths offer simplicity and lower communication, but face scalability issues. Microservices, on the other hand, provide better flexibility and scalability by supporting decoupling, but introduce complex communication overhead between services. Finally, the modular monolith combines the benefits of both architectures; however, once the services are distributed, the architecture faces similar communication challenges as microservices.

### *5.4. RQ4: What Frameworks Are Available in the Literature and Industry to Implement Modular Architecture?*

The distribution of the 12 studies that addressed MMA-related frameworks, tools, or implementation techniques is displayed in Figure 9. This number does not include the other 3 studies (of the 15 included in this review) because they did not mention any frameworks. In their study, the authors of [22] proposed a way for their architecture to be implemented and used the Spring framework to demonstrate its applicability. Manoppo and Istiono [33] also highlighted the importance of the Spring Boot Framework, which is usually used for developing modular monoliths because it allows for easy module separation and service encapsulation within a monolithic structure. While [17] does not mention dedicated frameworks for modular monolithic architecture; the paper uses Spring

Boot as a technology to implement the original monolith. The Spring monolith framework provides the decoupling of domain logic into modular and lightweight features, including dependency injection and RESTful APIs. This also supports transitions to microservices for improved scalability. The number of papers that mention each framework is shown in Figure 9.

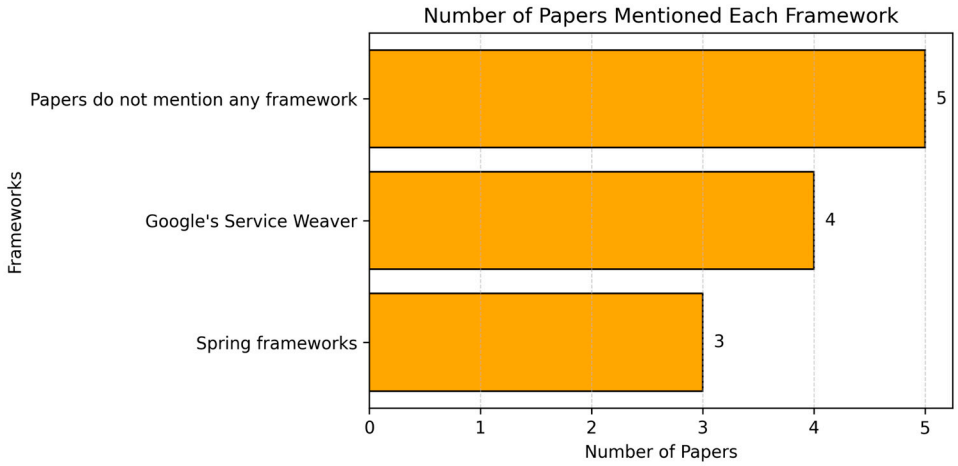


**Figure 9.** Number of papers mentioned each framework.

Barde [40] stated that Google's Service Weaver framework enables developers to write applications as a modular monolith and deploy them as microservices. Thus, the framework allows for component base separation while keeping a single codebase for the application. Apache Isis is also mentioned, as it is used to develop modular applications [40], and it is an open-source framework that uses DDD to allow the separation of business logic into distinct modules that can be individually maintained and tested.

As shown in Figure 10, the Service Weaver framework for developing modular monoliths in the cloud was also mentioned and discussed in [24,34,41]. This emerging framework aims to build a modular monolith that can be incrementally transitioned into microservices. This framework deploys components as microservices. Thus, it blends modularity and microservices advantages, facilitating performance and reducing communication overhead [35,45]. Figure 10 illustrates the foundation behind the Service Weaver framework.

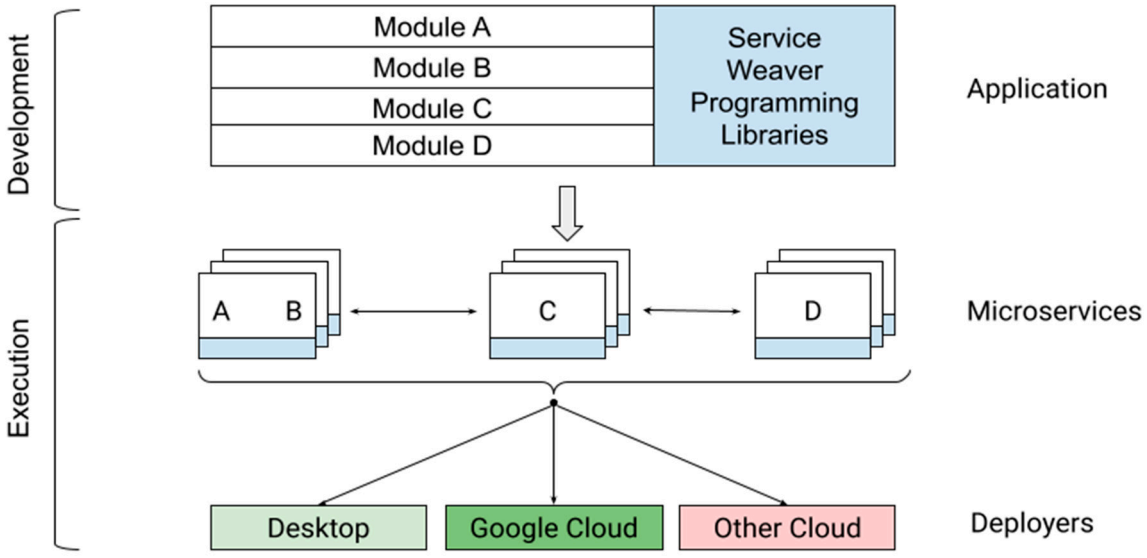


**Figure 10.** Google's Service Weaver framework [45].

### 5.5. RQ5: What Is the Impact on Performance, Maintainability, and Scalability When Migrating to Modular Monolith Architecture?

We found 14 empirical studies, including [17,22,24,33–35,39,41–43], and two non-empirical studies [33,38]. According to these empirical investigations, the modular monolith is regarded as a transitional structure that may help balance the difficulties of transitioning directly from a monolithic to a microservice architecture. Modular monoliths are less likely to experience the performance degradation that microservices bring about because of remote interaction and network overhead. The amount of data that is sent between modules, however, might provide issues for developers. However, we can confirm the performance advantage of MMA.

In terms of performance, a key advantage of adopting modular monolith is the reduced communication overhead when modules communicate within the same process. Communication happens through method calls rather than interservice (e.g., through a network), leading to lower latency and higher efficiency [24,40]. Furthermore, optimised access to domain entities significantly enhances performance in specific use cases by tightly integrating operations without needing external service calls [17,34,35]. Tsechelidis et al. [22] discussed that modular architecture enables inter-communications through method class rather than network-based communication, which leads to lower latency; this is a very important performance advantage in cloud computing. Gonçalves et al. [35] presented a case study in which they decomposed and obtained two types of modules: front-end and back-end modules. They also involved practical evaluations of performance before and after the migration, emphasising the benefits of optimised access using domain entities in the module architecture that leads to improved performance for certain functionalities. However, performance challenges arise in larger-scale systems, as inter-module communication can cause overhead [35], particularly when dealing with larger data volumes [17]. Moreover, poorly designed modular boundaries can introduce performance bottlenecks [38].

In terms of maintainability, the modular architecture provides a structure where the codebase is divided into dependent and domain-driven modules. This enhances maintainability by isolating changes in specific modules, reducing the need for wider system changes [22,33]. Furthermore, it facilitates a gradual improvement in system maintainability over time as modules are decoupled. However, in large-scale systems, complexity can be added to maintainability due to complex dependencies, especially when the number of modules and relationships between them grows [22,38]. This issue not only affects maintainability but performance as well, as bottlenecks become more apparent in larger systems due to the management of larger codebases and the overhead of modular inter-communication [22,32].

In terms of scalability, the adoption of the modular monolith can improve scalability compared to the traditional monolith due to its ability to independently scale different modules within the same system, which enables cloud systems to handle growing workloads without the need of scaling the entire application at once [24,30]. However, scalability challenges still exist when more complex and larger systems are at a scale where the inter-communication between modules introduces performance bottlenecks, which leads to not allowing the system to match the flexibility and distributed scalability at runtime [17,22,24,40,41]

## 6. Discussion

This section explains how MMA achieves a balance between modularity and cohesion in software architecture, how the design can change the way decisions are made in cloud environments, and how the results of this SLR are interpreted. Thus, this discussion

critically explores research implications, identifies practical opportunities, and future directions for both academia and industry.

### 6.1. Integration of Findings

Traditional modularity relies on either fully disconnected microservices or tightly coupled monoliths. MMA overcomes this inconsistency by demonstrating that logical modularity can result in many benefits if design is controlled. Our synthesis demonstrates that MMA is a confined modularity pattern with logical modules having defined domain boundaries within a single local process, rather than being a transitional stage before moving to microservices. This local cohesiveness enables the achievement of maintainability and performance without distribution overhead by redefining modularity as an independent domain.

As opposed to per-module elasticity and independent fault isolation, MMA prioritises cost-effectiveness, simplicity, and maintainability. In practice, a shortage of per-module scaling limits usage in highly elastic, broadly distributed systems, but empirically, in-process calls have reduced latency. This limitation encourages the use of MMA for medium-scale systems. When it comes to global-scale, distributed deployments, MMA is not recommended.

Research indicates that MMA acts as a re-centralisation strategy (when microservice fragmentation raises coordination cost) and as a decomposition overview (a stable staging point before extracting services). As a result, MMA is a continuous development of architecture rather than sequential transition from monolith to microservices.

Furthermore, most included empirical studies provided evidence on maintainability benefits. Clearer module boundaries, often implemented through DDD, improve code cohesion, testing, and refactoring. However, some studies reported challenges in achieving proper modularisation, such as hidden couplings and difficulty in defining bounded contexts. These issues suggest that MMA does not guarantee maintainability can be improved unless disciplined architectural practices are used.

Thus, MMA occupies a middle ground, as it avoids the operational complexity and orchestration overhead of microservices. However, this results in reduced scalability and resilience under high loads. Many included studies reported that MMA is often adopted as a transitional stage: either towards microservices or to re-centralise fragmented microservice systems. Thus, MMA may appear context-dependent in practice, rather than being a standalone architecture.

### 6.2. Research Implications, Opportunities and Future Research Directions

#### 6.2.1. Implications for Practice

The findings of our review provide practitioners with structured evidence to guide adoption decisions for MMA. Our synthesis shows that MMA should not be viewed as a replacement for either monoliths or microservices but rather as a context-dependent option situated between the two architectures.

MMA is useful in the following scenarios:

- The priority is for the operational simplicity and maintainability, because internal modularity supports clearer boundaries, reduced coupling, and easier refactoring. This is because having a domain well-bounded (DDD ready) has the potential to sustain cohesion and testing.
- The system domain is well-understood and can be partitioned through DDD principles.
- Cost efficiency is important, as MMA avoids the infrastructure and orchestration overhead which are main features of microservices.

- Scalability needs are of medium importance, which allows vertical scaling without the need for independent elasticity of services.
- Teams are DDD-mature and able to enforce modular boundaries effectively.

However, MMA may underperform in the following scenarios:

- Scenarios that require high elasticity and resilience, where independent scaling and fault isolation are critical.
- Scenarios that need global distribution, which requires autonomous services and diverse technology stacks.
- Scenarios with complex, large-scale systems, where it is difficult to identify and maintain modular boundaries.

In practice, MMA offers balance: it provides many of the modularity and maintainability benefits associated with microservices while maintaining the simplicity of a single deployable unit. For small to medium-scale projects, legacy system modernisation, or organisations seeking to reduce microservice fragmentation, MMA can be a good option. For large-scale, high-traffic, or globally distributed applications, microservices remain the more suitable choice. The immature frameworks that can support the architecture restrict the practical implementation of MMA. Although newer frameworks like Spring Modulith and Service Weaver make modular programming easier, they still lack established processes for fault isolation and testing.

#### 6.2.2. Opportunities and Future Research Directions

After conducting our SLR, we were able to identify several promising directions for future research on MMA in cloud environments. Particularly, we think that there should be a focus on three key areas of research:

1. Research on formalising the definition of MMA: The current literature contains inconsistencies in the terms used to refer to MMA. Many terms are used, such as "modular monolith," "modulith," and "modular monolithic architecture". Consensus on the scope and boundaries of such terms is required. Future research should focus on setting clear definitions, architectural taxonomies, and reference models to clearly distinguish MMA from both traditional monoliths and microservices.
2. Research on empirical evaluation and benchmarking: There is a scarcity of large-scale empirical studies that compare MMA to microservices and monoliths in terms of performance, scalability, maintainability, and cost. Future work should develop systematic benchmarks, case studies, and controlled experiments to evaluate trade-offs in real-world cloud environments. This is in line with the recommendations of [26], who recommended conducting quantitative analysis to systematically compare monoliths, microservices, and modular monoliths (moduliths) in terms of performance, scalability, and maintainability across diverse application domains and deployment scales.
3. Research on frameworks, tools, and migration strategies: The emerging frameworks, such as Spring Modulith and Service Weaver, support MMA. However, there is limited evidence of their adoption, limitations, and developer experience. Future research should explore tooling improvements, migration patterns from microservices or monoliths to MMA, and automation techniques for deployment and monitoring in cloud-native contexts.
4. Long-term performance and maintainability: Additional studies are needed to assess the long-term feasibility of MMA across different industries and domains. Comparative technical studies involving monolithic, modular monolithic, and microservices architectures could provide empirical evidence on how MMA performs over time, offering deeper insights to guide both academic discourse and organisational decision-making.

To conclude, the future of MMA in cloud environments holds significant promise. However, this review demonstrates that research in the area remains insufficient, with limited empirical evidence and fragmented conceptualisations. Advancing this field requires coordinated efforts to clarify definitions, build empirical knowledge, and refine supporting tools. Such work is critical to realising MMA's potential as a hybrid solution bridging the gap between monolithic and microservices architectures in both academic and industrial contexts.

### *6.3. Threats to Validity and Limitations*

This section highlights potential threats to the systematic literature review's validity [46].

#### 6.3.1. Internal Validity

The degree to which a study's main conclusion has been validated is known as internal validity. The possibility of bias among researchers conducting the SLR poses a risk to internal validity as it could impact the collection of data and study insights. To mitigate this threat, both authors were involved in the review process, including the selection of papers, the extraction of data, and analysis of the results. In each step, we applied cross-checking methods to minimise bias. Furthermore, we defined and used a clear review protocol that included well-established inclusion and exclusion criteria. We recommend that future replications should include multiple reviewers to strengthen reliability.

#### 6.3.2. External Validity

External validity refers to the extent to which the findings of the study can be generalised to the general field of modular monolithic architecture in a cloud environment. A threat to the external validity of this SLR is the potential to not include relevant primary studies. While we applied the search string to the main six digital libraries that publish work in computing and software, the lack of an automatic process to retrieve the studies would affect the reproducibility of the literature review [47]. Yet, we acknowledge that we may have missed primary studies. To minimise this threat to validity, we applied the snowballing process to the studies retrieved by the search string on the six main libraries to uncover additional works that might have been missed. For example, a potential threat to validity arises from the exclusion of the paper, [25]. Although this study addresses modular monolith migration, it emphasises cross-cloud migration strategies and cost optimisation trade-offs, rather than the architectural characteristics of MMA. As a result, it did not fully align with our inclusion criteria, in which we required studies to explicitly discuss architectural design, trade-offs, or frameworks. However, to mitigate this risk of overlooking relevant migration-related insights, we included two other papers by the same authors: [39] and [42]. These works directly examine modular monolithic applications (e.g., the Hermit Portal) and provide empirical evidence on migration challenges and architectural considerations, thereby ensuring that operational contexts were still represented without compromising the architectural focus of our review.

Furthermore, the lack of consistent terminology in the literature poses a big challenge. That is, while some studies use the term "modular monolith," while others refer to "modulith" or alternative modularisation concepts. We explicitly defined the term "modular monolithic architecture (MMA)" at the beginning of the review and used it consistently to avoid problems. However, differences in terminology among the retrieved studies and the selected primary studies may have limited the completeness of our search and synthesis. To mitigate this problem, we added all the identified terms to our search string to avoid missing related studies.

Finally, our review focused exclusively on peer-reviewed English-language publications. The exclusion of non-English literature and grey literature, such as technical reports,

industrial white papers, and blogs, may have omitted valuable practitioner insights into MMA adoption and real-world challenges. This restriction reduces the generalisability of our findings, particularly regarding industry practices, and excluding grey literature may have hindered the finding of key insights from real experiences on MMA. A further limitation is that, although some studies (e.g., [6,36]) explicitly discuss MMA, they were excluded because they were published in journals not indexed in Scopus. This may have led to the omission of potentially relevant insights; however, their exclusion was necessary to preserve the rigour of the review by restricting primary studies to reputable, indexed venues.

#### 6.3.3. Conclusion Validity

Based on what defines research in the subject of MMA, conclusion validity refers to whether we established the right parameters and whether we were able to achieve the appropriate measure. The quality of the chosen studies poses a risk to the validity of the conclusion; lower-quality studies could provide insights that are not justified or relevant to the MMA community. Short papers, demo papers, book chapters, reviews, and roadmap papers were not included in the study to minimise this risk. Additionally, we assessed each chosen paper's quality score.

Furthermore, during the screening process, we identified studies such as [37] that propose a large language model-based approach for decomposing monolithic applications into microservices. We decided to exclude this paper, although it is significant to the overall area of architectural engineering, because it does not explicitly discuss MMA, its trade-offs, or its application in cloud environments. The paper focused on enhancing decomposition using huge language models. By excluding such relevant studies, we guarantee that our review will remain consistent with the specified research questions and inclusion criteria.

#### 6.3.4. Reliability

Reliability is the degree to which this work can be replicated in a future investigation. To reduce this risk, our collected and categorised data have all been made accessible online. To ensure reproducibility, we also included a list of web sources, a specific search string, and other details in the research protocol. Another risk here is researcher bias, which could lead to similar findings if the systematic literature analysis were to be repeated with a new group of reviewers.

Finally, the relatively small number of included studies (15) limits the strength of general conclusions that can be drawn. Thus, our conclusions can be seen as indicative rather than definitive.

## 7. Conclusions

This SLR provides a critical evaluation of the fragmented body of knowledge on MMA in cloud environments. We systematically reviewed 15 primary studies across 6 digital libraries (2020–2025) and applied Kitchenham's SLR protocol with quality assessment. Thus, this study establishes the first comprehensive evidence base for MMA.

We provided a comprehensive definition of MMA, clarified its benefits and challenges, compared it with monolithic and microservices architectures, and identified the emerging tools and frameworks.

We concluded that MMA can be positioned as a hybrid and context-dependent architectural choice. Its architecture offers operational simplicity and modularity. Its scalability and resilience are also restricted when compared to a microservices architecture.

From an academic perspective, this study advances software architecture research because it resolves conceptual ambiguities in MMA terminology and scope, maps empir-

ical evidence of quality attributes such as performance, scalability, and maintainability, and outlines gaps that require future research, including empirical benchmarking and tools evaluation.

For practitioners, the results provide structured insights into situations when MMA's benefits outweigh its risks, particularly in scenarios that need maintainability, cost efficiency, and reduced operational overhead.

Overall, this review establishes a research agenda to guide the systematic development of MMA as an architectural paradigm and consolidates existing evidence. The review clarified definitions, identified empirical works, and aligned industry practices with academic discourse. Therefore, this study helps to position MMA as an evidence-informed alternative within the broader cloud-native software architecture landscape.

This systematic review reveals a significant gap in the literature regarding MMA. Only a few preliminary studies focus on the architecture itself, often describing an example or relying on a limited scope or relatively simple implementations and case studies. This limited available research shows that there are many research gaps and directions that can be addressed and highlights the need for comprehensive empirical research to thoroughly understand the architecture.

**Author Contributions:** Conceptualization, L.F.A.-Q. and A.A.-S.A.; methodology, L.F.A.-Q. and A.A.-S.A.; validation, L.F.A.-Q. and A.A.-S.A.; analysis, L.F.A.-Q. and A.A.-S.A.; data curation, L.F.A.-Q. and A.A.-S.A.; writing—original draft preparation, L.F.A.-Q.; writing—review and editing, L.F.A.-Q. and A.A.-S.A. All authors have read and agreed to the published version of the manuscript.

**Funding:** This research received no external funding.

**Data Availability Statement:** No new data was created or analysed in this study.

**Conflicts of Interest:** The authors declare no conflicts of interest.

## References

1. Rimal, B.P.; Jukan, A.; Katsaros, D.; Goeleven, Y. Architectural Requirements for Cloud Computing Systems: An Enterprise Cloud Approach. *J. Grid Comput.* **2011**, *9*, 3–26. [CrossRef]
2. Söylemez, M.; Tekinerdogan, B.; Tarhan, A.K. Challenges and Solution Directions of Microservice Architectures: A Systematic Literature Review. *Appl. Sci.* **2022**, *12*, 5507. [CrossRef]
3. Taibi, D.; Lenarduzzi, V.; Pahl, C. Processes, Motivations, and Issues for Migrating to Microservices Architectures: An Empirical Investigation. *IEEE Cloud Comput.* **2017**, *4*, 22–32. [CrossRef]
4. Taibi, D.; Systä, K. From Monolithic Systems to Microservices: A Decomposition Framework based on Process Mining. In Proceedings of the 9th International Conference on Cloud Computing and Services Science—CLOSER, Crete, Greece, 2–4 May 2019; SciTePress: Setúbal, Portugal, 2019; pp. 153–164. [CrossRef]
5. Bataieneh, S.; Ziadeh, A.; Al-Qora'N, L.F. Microservices Architecture for Improved Maintainability and Traceability in MVC-Based E-Learning Platforms: RoadMap for Future Developments. In Proceedings of the 2024 15th International Conference on Information and Communication Systems (ICICS), Irbid, Jordan, 13–15 August 2024; pp. 1–6. [CrossRef]
6. Mehta, G.; Pothineni, B.; Parthi, A.G.; Maruthavanan, D.; Veerapaneni, P.K.; Jayabalan, D.; Sankiti, S.R. Revisiting Monoliths: A Pragmatic Case for Transitioning from Microservices Back to Monolithic Architectures. *Int. J. Adv. Res. Comput. Commun. Eng.* **2024**, *13*, 328–337. Available online: https://www.researchgate.net/publication/387522735_Revisiting_Monoliths_A_Pragmatic_Case_for_Transitioning_from_Microservices_Back_to_Monolithic_Architectures (accessed on 22 October 2025).
7. Su, R.; Li, X.; Taibi, D. Back to the Future: From Microservice to Monolith. *arXiv* **2023**, arXiv:2308.15281. [CrossRef]
8. Oumoussa, I.; Saidi, R. Evolution of Microservices Identification in Monolith Decomposition: A Systematic Review. *IEEE Access* **2024**, *12*, 23389–23405. [CrossRef]
9. Su, R.; Li, X. Modular Monolith: Is This the Trend in Software Architecture? In Proceedings of the 1st International Workshop on New Trends in Software Architecture (SATrends '24). Association for Computing Machinery, New York, NY, USA, 14 April 2024; pp. 10–13. [CrossRef]
10. Su, R.; Li, X.; Taibi, D. From Microservice to Monolith: A Multivocal Literature Review. *Electronics* **2024**, *13*, 1452. [CrossRef]
11. Evans, E. *Domain-Driven Design Reference: Definitions and Pattern Summaries*; Dog Ear Publishing: Indianapolis, Indiana, 2014.

12. Rêgo, C.M.; Filho, R.C.M.; Mendonça, N.C. Performance and Resilience Impact of Microservice Granularity: An Empirical Evaluation Using Service Weaver and Amazon EKS. *Int. J. Netw. Manag.* **2025**, *35*, e70019. [CrossRef]
13. Kucukoglu, A. What Is Modular Monolith? 2023. Available online: https://serviceweaver.dev/ (accessed on 23 July 2024).
14. Parnas, D.L. On the criteria to be used in decomposing systems into modules. *Commun. ACM* **1972**, *15*, 1053–1058. [CrossRef]
15. Kitchenham, B. *Procedures for Performing Systematic Reviews*; Empirical Software Engineering National ICT Australia Ltd.: Eversleigh, Australia, 2004. Available online: https://www.researchgate.net/publication/228756057_Procedures_for_Performing_Systematic_Reviews (accessed on 22 October 2025).
16. Evans, E. *Domain-Driven Design: Tackling Complexity in the Heart of Software*; Addison-Wesley Professional: Boston, MA, USA, 2004.
17. Faustino, D.; Gonçalves, N.; Portela, M.; Silva, A.R. Stepwise migration of a monolith to a microservice architecture: Performance and migration effort evaluation. *Perform. Eval.* **2024**, *164*, 102411. [CrossRef]
18. Merson, P.; Yoder, J. Modeling Microservices with DDD. In Proceedings of the 2020 IEEE International Conference on Software Architecture Companion (ICSA-C), Salvador, Brazil, 16–20 March 2020; pp. 7–8. [CrossRef]
19. Felisberto, M. The Trade-Offs Between Monolithic vs. Distributed Architectures. 2024. Available online: https://arxiv.org/abs/2405.03619 (accessed on 1 September 2025).
20. Said, M.A.; Belouaddane, L.; Mihi, S.; Ezzati, A. Modulith Architecture: Adoption Patterns, Challenges, and Emerging Trends. *Int. J. Comput. Digit. Syst.* **2024**, *16*, 189–203.
21. Said, M.A.; Ezzati, A.; Mihi, S.; Belouaddane, L. Microservices Adoption: An Industrial Inquiry into Factors Influencing Decisions and Implementation Strategies. *Int. J. Comput. Digit. Syst.* **2024**, *15*, 1417–1432. [CrossRef]
22. Tsechelidis, M.; Nikolaidis, N.; Maikantis, T.; Ampatzoglou, A. Modular Monoliths the way to Standardization. In Proceedings of the ESAAM '23: Proceedings of the 3rd Eclipse Security, AI, Architecture and Modelling Conference on Cloud to Edge Continuum, Ludwigsburg, Germany, 17 October 2023; pp. 49–52. [CrossRef]
23. Poniszewska-Maranda, A.; MacIoch, J.; Borowska, B.; Maranda, W. Mechanisms for Transition from Monolithic to Distributed Architecture in Software Development Process. In Proceedings of the IEEE Computer Society's Annual International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunications Systems (MASCOTS), Houston, TX, USA, 3–5 November 2021; IEEE: New York, NY, USA, 2021; pp. 1–8. [CrossRef]
24. Ghemawat, S.; Grandl, R.; Petrovic, S.; Whittaker, M.; Patel, P.; Posva, I.; Vahdat, A. Towards Modern Development of Cloud Applications. In Proceedings of the HotOS 2023—Proceedings of the 19th Workshop on Hot Topics in Operating Systems, Providence, RI, USA, 22–24 June 2023; pp. 110–117. [CrossRef]
25. Olariu, F.; Alboaie, L.; Grămescu, R. Beyond Cloud Boundaries: An Analytical Case Study on the Migration of a Modular Monolith to Azure, AWS, and Google Cloud Platform. *Procedia Comput. Sci.* **2024**, *246*, 2782–2791. [CrossRef]
26. Prakash, C.; Arora, S. Systematic Analysis of Factors Influencing Modulith Architecture Adoption over Microservices. In Proceedings of the 2024 TRON Symposium (TRONSHOW), Tokyo, Japan, 11–13 December 2024; pp. 1–8.
27. Wohlin, C. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, in EASE '14, London, UK, 13–14 May 2014; Association for Computing Machinery: New York, NY, USA, 2014. [CrossRef]
28. Felizardo, K.R.; Mendes, E.; Kalinowski, M.; Souza, É.F.; Vijaykumar, N.L. Using Forward Snowballing to update Systematic Reviews in Software Engineering. In Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, in ESEM '16, Ciudad Real, Spain, 8–9 September 2016; Association for Computing Machinery: New York, NY, USA, 2016. [CrossRef]
29. Cabane, H.; Farias, K. On the impact of event-driven architecture on performance: An exploratory study. *Futur. Gener. Comput. Syst.* **2024**, *153*, 52–69. [CrossRef]
30. *ISO/IEC 25010:2011*; Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—System and Software Quality Models. International Organization for Standardization: Geneva, Switzerland, 2011.
31. Warrens, M.J. Five ways to look at Cohen's kappa. *J. Psychol. Psychother.* **2015**, *5*, 197. [CrossRef]
32. Hennink, M.M.; Kaiser, B.N.; Marconi, V.C. Code Saturation Versus Meaning Saturation: How Many Interviews Are Enough? *Qual. Health Res.* **2016**, *27*, 591–608. [CrossRef]
33. Manoppo, A.Y.P.; Istiono, W. Cloud-Based ERP System Backend Design, study case: PT Cranium Royal Aditama. *Ultim. J. Tek. Inform.* **2023**, *15*, 129–136. [CrossRef]
34. Johnson, J.; Kharel, S.; Mannamplackal, A.; Abdelfattah, A.S.; Cerny, T. Service Weaver: A Promising Direction for Cloud-Native Systems? In Proceedings of the International Conference on Cloud Computing and Services Science, CLOSER, Angers, France, 2–4 May 2024; pp. 167–175. [CrossRef]
35. Goncalves, N.; Faustino, D.; Silva, A.R.; Portela, M. Monolith Modularization towards Microservices: Refactoring and Performance Trade-offs. In Proceedings of the 2021 IEEE 18th International Conference on Software Architecture Companion, ICSA-C 2021, Stuttgart, Germany, 22–26 March 2021; IEEE: New York, NY, USA, 2021; pp. 54–61. [CrossRef]

36. Prakash, C. Architectural Trade-Offs in Modulith Architecture: A Case Study on Dependency and Data Management in Rewards Systems. *Int. J. Comput. Appl.* **2025**, *186*, 975–8887. [CrossRef]
37. Sellami, K.; Saied, M.A. Contrastive Learning-Enhanced Large Language Models for Monolith-to-Microservice Decomposition. *arXiv* **2025**, arXiv:2502.046042025. [CrossRef]
38. Shablii, T.; Tytenko, S. Modular Monolith As a Microservices Precursor. *Mod. Eng. Innov. Technol.* **2023**, *1*, 25–32. [CrossRef]
39. Olariu, F. Overcoming Challenges in Migrating Modular Monolith from On-Premises to AWS Cloud. In Proceedings of the RoEduNet IEEE International Conference, Chișinău, Moldova, 21–22 September 2023; IEEE: New York, NY, USA, 2023; pp. 1–6. [CrossRef]
40. Barde, K. Modular Monoliths: Revolutionizing Software Architecture for Efficient Payment Systems in Fintech. *Int. J. Comput. Trends Technol.* **2023**, *71*, 20–27. [CrossRef]
41. Mendonça Filho, R.C.; Mendonça, N.C. *Performance Impact of Microservice Granularity Decisions: An Empirical Evaluation Using the Service Weaver Framework BT—Software Architecture*; Galster, M., Scandurra, P., Mikkonen, T., Oliveira Antonino, P., Nakagawa, E.Y., Navarro, E., Eds.; Springer: Cham, Switzerland, 2024; pp. 191–206.
42. Olariu, F.; Alboaie, L. Challenges In Optimizing Migration Costs From On-Premises To Microsoft Azure. *Procedia Comput. Sci.* **2023**, *225*, 3362–3371. [CrossRef]
43. Bueno, A.; Díaz, M.; Rubio, B.; Martín, C. Chopping Off Iot Cloud Costs: A Case Study. Available SSRN 5111486. 2025. Available online: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5111486 (accessed on 1 September 2025). [CrossRef]
44. Guaman, D.; Yaguachi, L.; Samanta, C.C.; Danilo, J.H.; Soto, F. Performance evaluation in the migration process from a monolithic application to microservices. In Proceedings of the 2018 13th Iberian Conference on Information Systems and Technologies (CISTI), Caceres, Spain, 13–16 June 2018; pp. 1–8. [CrossRef]
45. Google. Introducing Service Weaver: A Framework for Writing Distributed Applications. Google Open Source Blog. Available online: https://opensource.googleblog.com/2023/03/introducing-service-weaver-framework-for-writing-distributed-applications.html (accessed on 9 September 2024).
46. Ampatzoglou, A.; Bibi, S.; Avgeriou, P.; Verbeek, M.; Chatzigeorgiou, A. Identifying, categorizing and mitigating threats to validity in software engineering secondary studies. *Inf. Softw. Technol.* **2019**, *106*, 201–230. [CrossRef]
47. Li, Z.; Rainer, A. Reproducible Searches in Systematic Reviews: An Evaluation and Guidelines. *IEEE Access* **2023**, *11*, 84048–84060. [CrossRef]